

# Compiladores

## Linguagem $\text{BML}_{26}$

Departamento de Eletrónica, Telecomunicações e Informática  
Universidade de Aveiro

Abril de 2026

---

### Objetivos

O objectivo geral deste trabalho é a definição formal da linguagem de programação  $\text{BML}_{26}$  (*Bit Manipulation Language, 2026*) e o desenvolvimento das ferramentas de suporte que permitam a tradução de uma descrição em  $\text{BML}_{26}$  para a linguagem de programação C++. Deverá ainda haver suporte para uma linguagem secundária, designada  $\text{BEC}$  ( *$\text{BML}_{26}$  Expressions Calculus*), que permita interpretar **em tempo de execução** um texto que represente uma expressão em  $\text{BML}_{26}$  e determine o seu valor.

### Tarefas

A concretização do objetivo geral deste projeto deve ser decomposto em:

- Definição formal da linguagem principal  $\text{BML}_{26}$ , associado a ficheiros com a extensão `bml`. O resultado desta tarefa deve ser a descrição em ANTLR de uma gramática que capte adequadamente a natureza sintática de descrições em  $\text{BML}_{26}$ .
- Construção de um analisador semântico que detete e sinalize erros de descrição  $\text{BML}_{26}$  que escapam à análise sintática. Compete a este analisador verificar se as variáveis são declaradas adequadamente antes de serem utilizadas, se há repetições de identificadores na declarações de variáveis, se as atribuições e operações são válidas, etc. O resultado desta tarefa deve ser o código fonte (*visitor*), na linguagem Java, do analisador semântico desenvolvido, assim como a sua adequada invocação no programa principal.
- Construção de um compilador que permite transformar uma descrição  $\text{BML}_{26}$  num programa equivalente na linguagem C++. Assume-se que a arquitetura destino não possui suporte hardware para vírgula flutuante, pelo que o código gerado na manipulação de vírgula fixa deve exclusivamente usar expressões inteiras. Encoraja-se e valoriza-se fortemente o uso de *string templates*. O resultado desta tarefa deve ser o código fonte do compilador (*visitor*, outro que não o do analisador semântico), na linguagem Java, assim como a sua adequada invocação no programa principal, e o ficheiro (`.stg`) com os *string templates* utilizados.
- Definição formal da linguagem secundária  $\text{BEC}$ . O resultado desta tarefa deve ser a descrição em ANTLR da gramática que represente as expressões.
- Construção de um interpretador que faça a validação semântica de expressões e calcule o seu valor. O resultado desta tarefa deve ser o código fonte (*visitor*), necessariamente na linguagem C++, do interpretador, assim como a sua adequada intergração na linguagem principal.

### Descrição das linguagens

Essencialmente, a perceção da sintaxe das construções gramaticais da linguagem  $\text{BML}_{26}$  deve ser extraída dos exemplos fornecidos. No entanto, apresentam-se aqui alguns aspetos considerados importantes, relacionados com o domínio específico de aplicação.

## Os tipos de dados inteiro e real

O tipo de dados inteiro é nativo em BML<sub>26</sub> e está associado a palavras-chave do tipo `s«N»`, para grandezas inteiras com sinal, e `u«N»`, para grandezas sem sinal, em que `«N»` representa a dimensão da representação, podendo ter os valores `8`, `16`, `32` e `64`. As grandezas negativas são representadas em complemento para 2.

Considera-se que as arquiteturas-alvo não possuem unidade de vírgula flutuante, pelo que a representação de números reais em vírgula flutuante não é suportada pela linguagem BML<sub>26</sub>. Em consequência, o código C++ gerado não pode conter constantes reais nem variáveis dos tipos `float` ou `double`. Os números reais são representados em vírgula fixa, considerando-se virtualmente a existência do separador fracional sobre uma grandeza inteira. Assim, o tipo de dados `u32.8` representa uma grandeza real, cujas partes inteira e fracionária ocupam, respetivamente, 24 e 8 bits.

A linguagem BML<sub>26</sub> suporta as operações aritméticas habituais sobre valores inteiros e reais. No entanto, é preciso ter presente que a implementação de aritmética sobre números reais pode obrigar ao alinhamentos de operandos. Por exemplo, se `n1` for do tipo `32.10`, `n2` do tipo `16.5` e `n3` do tipo `32.10`, a operação

```
n3 = n1 + n2;
```

obriga a que o valor de `n2` seja convertido para o formato `32.10`, antes que a adição seja feita.

A linguagem BML<sub>26</sub> suporta as operações lógicas bit-a-bit (*bitwise*) habituais sobre valores inteiros.

## Manipulação de bits

Em BML<sub>26</sub>, os valores inteiros podem ser encarados como arrays de bits, permitindo a manipulação destes de forma individual. Por exemplo, se `n1` for uma variável inteira de 32 bits, a instrução

```
n1[5] = 1;
```

coloca o bit 5 de `n1` a 1, mantendo os restantes bits inalterados. Bits contíguos podem ser manipulados em conjunto. Por exemplo,

```
n1[1..4] = 0xa;
```

coloca a 1 os bits 4 e 2 de `n1` e a 0 os bits 3 e 1, mantendo os restantes bits inalterados.

Em BML<sub>26</sub>, as operações booleanas apenas são aplicáveis a bits, manipulados individualmente. A aplicação a sequências de bits das operações aritméticas e das operações lógicas bit-a-bit deve ser analisada, devendo ser implementado o que se considere pertinente.

## Outras características da linguagem BML<sub>26</sub>

Tal como uma linguagem de programação genérica, BML<sub>26</sub> possui estruturas gramaticais que suportam a elaboração de programas podendo conter comentários de linha e de bloco, declaração de variáveis e de constantes, controlo de fluxo repetitivo e condicional, escrita em e leitura de ficheiros, entre outros.

## Sobre a linguagem secundária

Um programa (descrição textual) na linguagem secundária (BEC) representa uma expressão da linguagem BML<sub>26</sub>. Pode conter variáveis, que não devem ter nenhum efeito no código BML<sub>26</sub> que o executa (e vice-versa) e que se extinguem com a interpretação do programa (cálculo da expressão).

## Níveis de implementação da solução

As características que a solução pode contemplar estão distribuídas por 3 níveis, cada uma deles cobrindo as vertentes referidas acima (gramática, verificação semântica, geração de código e linguagem secundária).

- Nível mínimo – primeiro patamar de características que qualquer solução deve implementar.

- Nível desejável – características adicionais às de nível mínimo que a solução deve desejavelmente implementar. Este nível apenas é considerado para avaliação se a totalidade das características do nível mínimo tiverem sido implementadas.
- Nível avançado – características adicionais às de nível desejável, não diretamente definidas nesta proposta, mas que sejam pertinentes e de excelência para a linguagem BML<sub>26</sub>, que a solução pode implementar. Este nível apenas é considerado para avaliação se a totalidade das características dos níveis mínimo e desejável tiverem sido implementadas.

## Nível mínimo

Os exemplos que começam por `min ( min-?? .bml )` correspondem a código fonte que tem de ser aceite (e devidamente compilado e interpretado) pelas linguagens a desenvolver. Incluem:

- Os tipos de dados: inteiro, com e sem sinal, de 8, 16 e 32 bits; real, representado em vírgula fixa; texto.
- Expressões aritméticas e lógicas bit-a-bit (*bitwise*) sobre grandezas inteiras; expressões aritméticas sobre grandezas reais; concatenação de texto; operadores de conversão entre tipos; respeito pela precedência habitual dos vários operadores envolvidos. (Use a precedência dos operadores do C/C++ como referência.)
- Escrita de texto no *standard output*; leitura de texto do *standard input*.
- Interpretação em tempo de execução de expressões a partir de texto que as represente.

## Nível desejável

Os exemplos que começam por `des ( des-?? .bml )` incluem código fonte que se enquadra nas características desejáveis. Incluem:

- A manipulação direta de bits e de sequências de bits de um inteiro.
- Expressões booleanas (predicados) contendo, pelo menos, relações de ordem e operadores booleanos (conjunção, disjunção, etc.).
- Instrução condicional (operando sobre expressões booleanas).
- Instruções iterativas.
- Manipulação de arrays.
- Escrita no *standard error* e em ficheiros; leitura de ficheiros.

## Nível avançado

Neste nível, compete ao grupo identificar, propor e implementar características que possam enriquecer as linguagens BML<sub>26</sub> e BEC, desde que, tal como referido anteriormente, sejam pertinentes e de significativa mais valia.